

MobAuditFlow: An AI-Powered Automated Mobile Security Audit Platform Based on OWASP MASVS/MASTG

Yousra Zarri^a, Malak Belkho^a, Mounir Merghich^a, Laazouzi Hiba^a

^aNational School of Applied Sciences (ENSA), Marrakech, Cadi Ayyad University, Morocco

Preprint submitted May 25, 2026

Abstract

Mobile application security auditing is a time-consuming and expertise-intensive process that traditionally relies on manual analysis of static scanning results against the OWASP Mobile Application Security Verification Standard (MASVS). MobAuditFlow is an intelligent, automated audit platform that combines static APK analysis, AI-powered vulnerability mapping, Retrieval-Augmented Generation (RAG), and document automation to transform raw MobSF scan outputs into structured, professional-grade PDF security reports. The platform orchestrates a multi-agent AI pipeline via n8n, using Google Gemini for cloud-based reasoning and a locally deployed Ollama model connected through a RAG pipeline backed by ChromaDB and SentenceTransformers for privacy-preserving, context-aware MASVS compliance analysis. The RAG component, developed as a standalone FastAPI microservice, embeds all 24 MASVS controls as vector representations and retrieves the most semantically relevant controls for any given finding. Results on OWASP UnCrackable-Level1 demonstrate that the platform correctly identifies and maps common Android vulnerabilities including hardcoded secrets, insecure storage, weak cryptography, and root detection evasion. MobAuditFlow significantly reduces the manual workload of security auditors while producing contextualised, actionable remediation recommendations aligned with OWASP standards.

Keywords: Mobile Security, OWASP MASVS, RAG, Automated Auditing, Android, AI, n8n, ChromaDB, MobSF, Ollama



Figure 1: MobAuditFlow platform overview.

Code and Software Metadata

Table 1: Software metadata

Nr.	Description	Value
C1	Current code version	1.0.0
C2	Repository	https://github.com/Mounir505/MobAuditFlow
C3	Legal Code License	MIT License
C4	Code versioning system	Git
C5	Languages, tools, services	Python, FastAPI, ChromaDB, SentenceTransformers, Ollama, n8n, Google Gemini API, MobSF, Gotenberg, Docker, JavaScript, Node.js
C6	Dependencies	Python 3.10+, Docker, n8n, Ollama, ChromaDB, uvicorn, ngrok
C7	Support email	y.zarri7127@uca.ac.ma

1 Motivation and Significance

The security of mobile applications has become a critical concern as Android applications increasingly handle sensitive user data including authentication credentials, financial information, and personal health records. The OWASP Mobile Application Security Verification Standard (MASVS) [1] and its companion testing guide (MASTG) [2] define the industry-accepted framework for evaluating mobile application security across categories including storage, cryptography, authentication, network communication, platform interaction, code quality, and resilience.

In practice, conducting a complete MASVS-compliant audit of a single Android application is a highly manual process. Security analysts must run static and dynamic analysis tools such as MobSF [3], interpret large volumes of JSON output, manually identify which MASVS controls are violated by each finding, filter false positives, and produce structured reports documenting vulnerabilities and remediation recommendations. This workflow is both time-intensive and requires expert-level knowledge of Android internals, reverse engineering, and OWASP standards.

Existing automation efforts in mobile security focus primarily on individual stages of this pipeline — MobSF handles static scanning, while tools like QARK or AndroBugs cover specific vulnerability classes — but none provide an end-to-end pipeline from raw scan output to a structured, MASVS-mapped, AI-enriched security report. Furthermore, most current approaches rely on cloud-based AI APIs, raising data confidentiality concerns in enterprise contexts.

MobAuditFlow addresses these gaps by providing a fully orchestrated, modular platform that automates the entire audit workflow: from APK ingestion and static analysis through AI-powered vulnerability mapping, RAG-based MASVS compliance checking, and automated PDF report generation. The inclusion of a locally-deployed LLM via Ollama ensures that sensitive APK contents never leave the organisation’s infrastructure.

1.1 Software Architecture

MobAuditFlow follows a distributed, microservice-based architecture in which each component has a clearly defined responsibility. The central orchestration engine is n8n, a low-code workflow automation platform, which coordinates the flow of data between all system components.

The architecture is composed of the following layers:

- **Ingestion Layer:** Google Drive acts as the entry point. A Google Drive Trigger node in n8n monitors a designated folder and automatically initiates the audit pipeline whenever a new APK file is deposited. The file is downloaded and forwarded to the MobSF Uploader node, which submits it to the MobSF Scanner for static analysis.

- **Scanning Layer:** The Scanner node submits the APK to MobSF and initiates a full static analysis. A Wait node ensures the pipeline pauses until the scan is complete before proceeding. The Scorecard node then retrieves the structured JSON findings from MobSF and passes them downstream.
- **AI Analysis Layer:** The Scorecard output is distributed in parallel to four specialised agents:
 - **Ollama (RAG):** An HTTP Request node that calls the locally-deployed FastAPI RAG endpoint. It performs MASVS/MASTG-grounded analysis, assigns CVSS v3.1 scores, and retrieves NVD CVE references.
 - **AI Agent 1 — Network/API:** A Gemini-powered agent analysing TLS configuration, cleartext traffic, and certificate validation issues.
 - **AI Agent 2 — Manifest:** A Gemini-powered agent inspecting AndroidManifest.xml for dangerous permissions, exported components, and debug flags.
 - **AI Agent 3 — Reverse Engineering:** A Gemini-powered agent detecting hardcoded secrets, weak obfuscation, and cryptographic weaknesses.
- **Aggregation Layer:** A Merge node collects and synchronises the outputs of all four parallel agents, configured in *Wait for all inputs to arrive* mode to ensure no partial report is generated. A CVSS Summary node then computes the overall risk score (maximum and average CVSS), counts findings by severity, and generates the executive summary HTML block passed to the report generator.
- **Report Generation Layer:** A Code in JavaScript node assembles all agent outputs, the executive summary, and the RAG section into a structured HTML document. Gotenberg [7] converts the HTML to a professional PDF report, which is then automatically dispatched to the designated recipient via the Send a Message node.

The complete n8n orchestration workflow is shown in Figure 3, illustrating the full pipeline from Google Drive trigger through parallel AI analysis to final PDF report delivery.

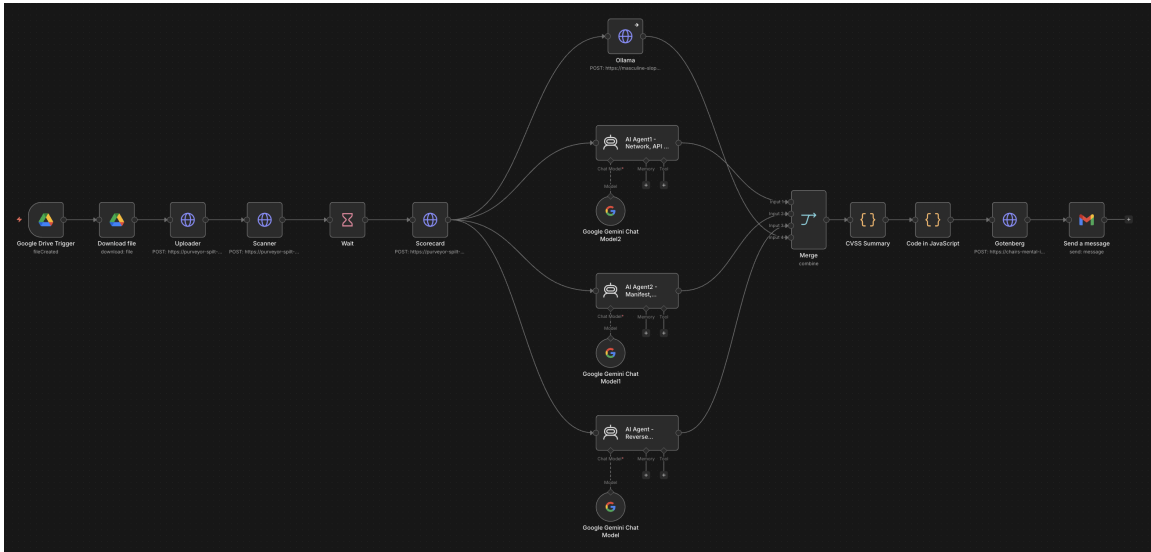


Figure 2: MobAuditFlow system architecture.

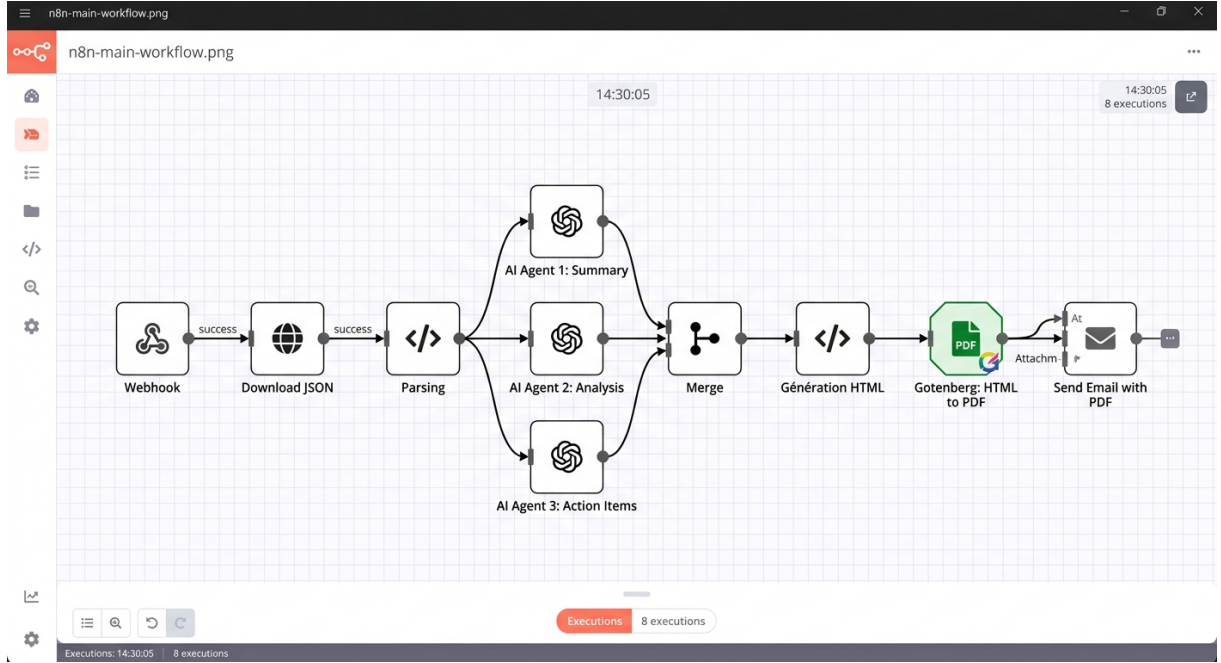
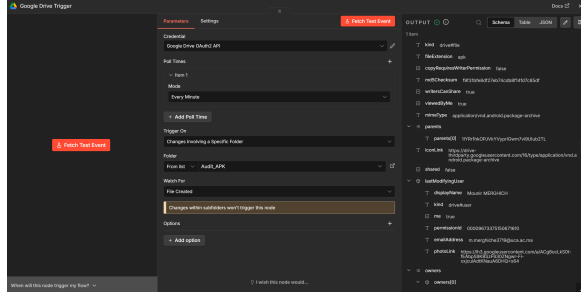
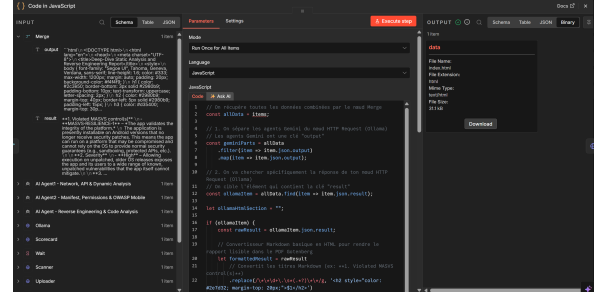


Figure 3: n8n main orchestration workflow.

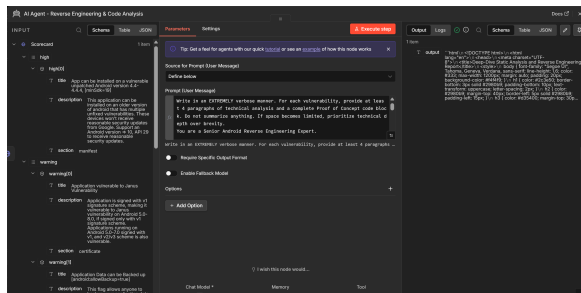
To provide further visibility into the pipeline internals, Figure 4 illustrates the four principal n8n nodes that drive the automation: the Google Drive trigger that initiates the pipeline, the JavaScript node responsible for JSON preprocessing, the Gemini-powered AI agent, and the Merge node that synchronises parallel agent outputs before report generation.



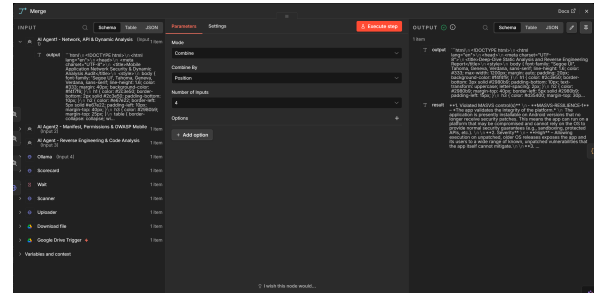
(a) Google Drive trigger node.



(b) JavaScript preprocessing node.



(c) Gemini AI agent node.



(d) Merge node configuration.

Figure 4: Key n8n workflow nodes.

1.2 Component Summary

Table 2: MobAuditFlow system components and their roles

Component	Role
MobSF	Static and dynamic APK analysis; produces JSON vulnerability findings
n8n	Central workflow orchestrator; coordinates all pipeline stages
Google Gemini API	Cloud LLM for manifest, network, and reverse engineering agent reasoning
Ollama (local LLM)	Privacy-preserving local inference for RAG-augmented MASVS analysis
ChromaDB	Vector database storing MASVS control embeddings for semantic retrieval
SentenceTransformers	Embedding model (<code>all-MiniLM-L6-v2</code>) for encoding findings and MASVS controls
FastAPI	REST API exposing the RAG pipeline to n8n
ngrok	Secure tunnel exposing the local FastAPI service to the n8n cloud instance
Gotenberg	HTML-to-PDF conversion for professional report generation
Docker	Containerisation of MobSF, ChromaDB, Gotenberg, and n8n services
Google Drive	APK scan file ingestion point and trigger

1.3 Multi-Agent Architecture

The AI analysis layer employs four specialised agents operating in parallel, each targeting a distinct security domain:

Table 3: Specialised AI agents and their responsibilities

Agent	Responsibility
Agent 1 — Network/API	Analyses TLS configuration, cleartext traffic, insecure end-point usage, and certificate validation issues
Agent 2 — Manifest	Inspects AndroidManifest.xml for dangerous permissions, exported components, and debug flags
Agent 3 — Reverse Engineering	Detects hardcoded secrets, weak obfuscation, and sensitive data exposure in decompiled code
Agent 4 — Ollama/RAG	Performs MASVS-grounded analysis using the local RAG pipeline for privacy-sensitive findings

All agents output structured JSON responses that are collected by the n8n Merge node, configured in *Wait for all inputs to arrive* mode to ensure report coherence. Figure 5 shows the Ollama RAG agent node as configured within the n8n workflow.

2 Comparative Analysis: MobSF and MobAuditFlow

MobSF is a widely adopted open-source static and dynamic analysis framework for Android and iOS applications. While MobSF provides a comprehensive set of automated scanning capabilities, it operates as a standalone analysis tool and does not address the downstream requirements of a complete security audit workflow. MobAuditFlow positions itself not as a replacement for MobSF but as an orchestration layer that consumes MobSF output and transforms it into a fully enriched, standards-aligned audit report through AI reasoning, OWASP knowledge retrieval, risk quantification, and automated documentation.

Table 4 presents a detailed feature comparison between MobSF and MobAuditFlow across the dimensions most relevant to a professional mobile security audit.

Table 4: Feature comparison between MobSF and MobAuditFlow

Feature	MobSF	MobAuditFlow
APK static analysis	Yes	Yes (uses MobSF internally)
Dynamic analysis	Yes	Not yet (future work)
iOS IPA support	Yes	Not yet (future work)
Output format	JSON + web dash-board	Professional PDF report
MASVS v2 control mapping	No	Automatic via RAG
MASTG verification procedures	No	Yes (ChromaDB retrieval)
CVSS v3.1 risk scoring	No	Automatic per finding
CVE references (NVD)	No	Yes (real-time API lookup)
AI-powered analysis	No	Yes (Gemini + Ollama)
RAG knowledge base	No	24 MASVS + 50+ MASTG docs
Multi-agent reasoning	No	Yes (4 specialised agents)
False positive filtering	No	Designed and validated
Executive summary	No	Yes (CVSS-based risk score)
MASVS compliance matrix	No	Yes (24 controls PASS/FAIL)
Automated report delivery	No	Yes (email via n8n)
Privacy-preserving local AI	N/A	Yes (Ollama, no cloud)
Workflow automation	No	Yes (n8n orchestration)
Requires security expertise	Yes (to interpret)	No (self-explanatory report)
Target audience	Security experts	Security teams, managers, clients

The fundamental distinction between the two tools can be summarised as follows: MobSF detects vulnerabilities and presents raw technical evidence; MobAuditFlow analyses, contextualises, prioritises, and documents those vulnerabilities in a format directly usable by security teams, project managers, and clients. MobSF requires a qualified analyst to interpret its output and manually produce a report; MobAuditFlow automates this entire post-scan workflow without human intervention.

From an academic perspective, the key novelty of MobAuditFlow lies in the integration of RAG-augmented AI reasoning with OWASP standards — a combination that addresses the hallucination problem inherent in general-purpose LLMs when applied to security tasks, while simultaneously providing verification procedures, quantified risk scores, and real-world CVE evidence that no existing automated mobile security tool currently offers in a single unified pipeline.

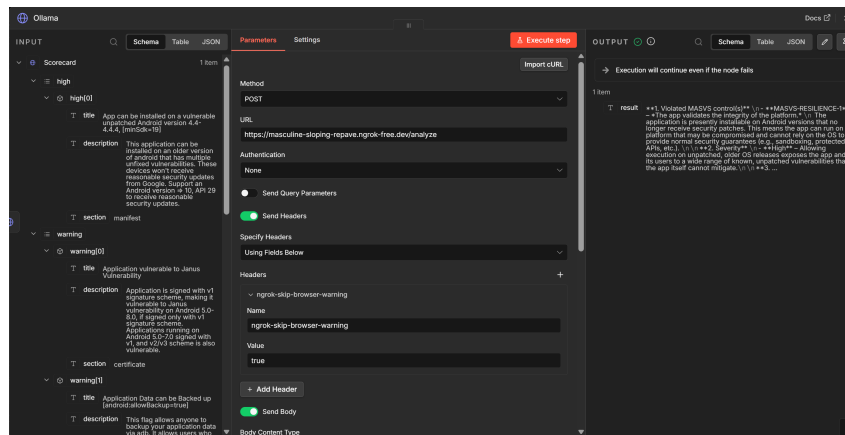


Figure 5: Ollama RAG agent node.

3 The RAG Component: MASVS/MASTG-Aware AI Analysis

3.1 Overview and Role

The RAG (Retrieval-Augmented Generation) component was designed and implemented by Yousra Zarri as the P2 contribution to MobAuditFlow. Its purpose is to ground AI-generated security analysis in authoritative OWASP documentation, thereby reducing hallucinations and ensuring that every finding is mapped to a specific, verifiable MASVS control with associated MASTG verification procedures, CVSS risk scoring, and real-world CVE references.

The component evolved through several iterations. The initial version embedded only the 24 MASVS controls. Subsequent improvements added MASTG test procedure ingestion, CVSS v3.1 scoring, and NVD CVE lookup, substantially enriching the analytical output returned to the n8n pipeline.

3.2 MASVS and MASTG Knowledge Base Ingestion

The OWASP MASVS repository [1] and MASTG repository [2] were cloned locally. The MASVS controls/ directory contains 24 Markdown files, each corresponding to one security control. The MASTG tests/android/ directory contains over 50 test procedure files organised by MASVS category.

Each document was embedded using the all-MiniLM-L6-v2 SentenceTransformer model [4] and stored in a persistent ChromaDB [5] instance as two separate collections: masvs and mastg. The combined ingestion script is shown in Listing 1.

Listing 1: Combined MASVS and MASTG ingestion script (ingest_all.py)

```

1 import os, chromadb
2 from sentence_transformers import SentenceTransformer
3
4 model = SentenceTransformer("all-MiniLM-L6-v2")
5 client = chromadb.PersistentClient(path="./masvs_db")
6 masvs_col = client.get_or_create_collection("masvs")
7 mastg_col = client.get_or_create_collection("mastg")
8
9 # Ingest MASVS controls
10 for filename in os.listdir("./owasp-masvs/controls"):
11     if filename.endswith(".md"):
12         with open(f"./owasp-masvs/controls/{filename}") as f:
13             content = f.read()
14             control_id = filename.replace(".md", "")
15             masvs_col.add(
16                 documents=[content],
17                 embeddings=[model.encode(content).tolist()],
18                 ids=[control_id],
19                 metadatas=[{"control": control_id, "type": "masvs"}]
20             )
21
22 # Ingest MASTG test procedures
23 mastg_base = "./owasp-mastg/tests/android"
24 for category in os.listdir(mastg_base):
25     cat_path = os.path.join(mastg_base, category)
26     if not os.path.isdir(cat_path):
27         continue
28     for filename in os.listdir(cat_path):
29         if filename.endswith(".md"):
30             with open(os.path.join(cat_path, filename)) as f:
31                 content = f.read()
32                 test_id = filename.replace(".md", "")
33                 mastg_col.add(
34                     documents=[content],
35                     embeddings=[model.encode(content).tolist()],
36                     ids=[test_id],
37                     metadatas=[{"test": test_id,
38                                "category": category,

```



```

39         "type": "mastg" }]]
40     )

```

3.3 RAG Query Pipeline

When the n8n workflow sends a MobSF finding to the RAG endpoint, the system performs the following steps:

1. The finding text is encoded into a vector embedding using `all-MiniLM-L6-v2`.
2. ChromaDB performs a cosine similarity search against the `masvs` collection to retrieve the top-3 most relevant MASVS controls.
3. ChromaDB performs a second cosine similarity search against the `mastg` collection to retrieve the top-2 most relevant Android test procedures.
4. A structured prompt combining the finding, MASVS context, and MASTG context is sent to the locally deployed Ollama LLM.
5. The model returns a structured analysis identifying the violated control(s), severity level, MASTG-based verification steps, and remediation recommendations.
6. The MASVS control ID is extracted from the response and passed to the CVSS scoring module.
7. A CVSS v3.1 score, severity rating, and vector string are retrieved from the control-to-CVSS lookup table.
8. The NVD public API is queried to retrieve real CVE references associated with the identified control.
9. All results are returned as a structured JSON response to n8n.

3.4 CVSS v3.1 Scoring

Each MASVS control is mapped to a pre-defined CVSS v3.1 base score based on the typical impact profile of vulnerabilities in that category. The mapping is implemented in `cvss.py` and called by `api.py` after control identification. Table 5 presents a representative subset of the mapping.

Table 5: Representative MASVS-to-CVSS v3.1 mapping

MASVS Control	Score	Rating	CVSS Vector
MASVS-STORAGE-1	7.5	High	AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
MASVS-CRYPTO-1	7.4	High	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N
MASVS-AUTH-1	8.1	High	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
MASVS-NETWORK-1	7.4	High	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N
MASVS-CODE-2	7.5	High	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
MASVS-RESILIENCE-1	6.2	Medium	AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

3.5 CVE Enrichment via NVD

The `cve_lookup.py` module queries the National Vulnerability Database (NVD) REST API [?] using keywords derived from each MASVS control. For example, MASVS-STORAGE-1 maps to the query “*android plaintext storage credentials*”, returning matching CVEs with their identifiers, CVSS scores, severity ratings, descriptions, and direct NVD links. This connects each audit finding to its real-world exploit history, significantly enriching the analytical value of the report.

3.6 FastAPI Service

The complete RAG pipeline is exposed as a REST API via FastAPI [6]. The service implements lazy model loading to avoid startup latency and exposes two endpoints:

- POST /analyze — accepts {"finding": "..."} and returns the full enriched analysis
- GET /health — returns service status for monitoring

The complete response structure returned by /analyze is:

Listing 2: FastAPI /analyze endpoint response structure

```

1 return {
2     "result":      ai_result,      # Ollama analysis text
3     "control":     control_id,     # e.g. MASVS-STORAGE-1
4     "cvss_score":  cvss["score"],  # e.g. 7.5
5     "cvss_vector": cvss["vector"], # CVSS v3.1 vector string
6     "cvss_rating": cvss["rating"], # High / Medium / Low
7     "related_cves": cves           # list of NVD CVE objects
8 }

```

The service is launched with uvicorn and exposed publicly via ngrok, providing the n8n Ollama node with a stable HTTPS endpoint.

3.7 Example Output

Table 6 illustrates the enriched RAG component output for a representative MobSF finding, demonstrating the full pipeline from raw finding to structured analysis with CVSS scoring and CVE references.

Table 6: Enriched RAG analysis output for a plaintext credential storage finding

Field	Value
Input Finding	<i>"The app stores user credentials in SharedPreferences in plaintext"</i>
Retrieved MASVS Controls	MASVS-STORAGE-1, MASVS-STORAGE-2, MASVS-AUTH-2
Retrieved MASTG Tests	MASTG-TEST-0001 (MASVS-STORAGE), MASTG-TEST-0003 (MASVS-STORAGE)
Violated Control	MASVS-STORAGE-1
Severity	High
CVSS Score	7.5 (AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
MASTG Verification	Inspect SharedPreferences XML via ADB; check for EncryptedSharedPreferences usage
Recommended Fix	Use Android Keystore or EncryptedSharedPreferences; never persist raw passwords
Related CVEs	CVE-2021-0646 (CVSS 7.8), CVE-2020-0082 (CVSS 7.5)

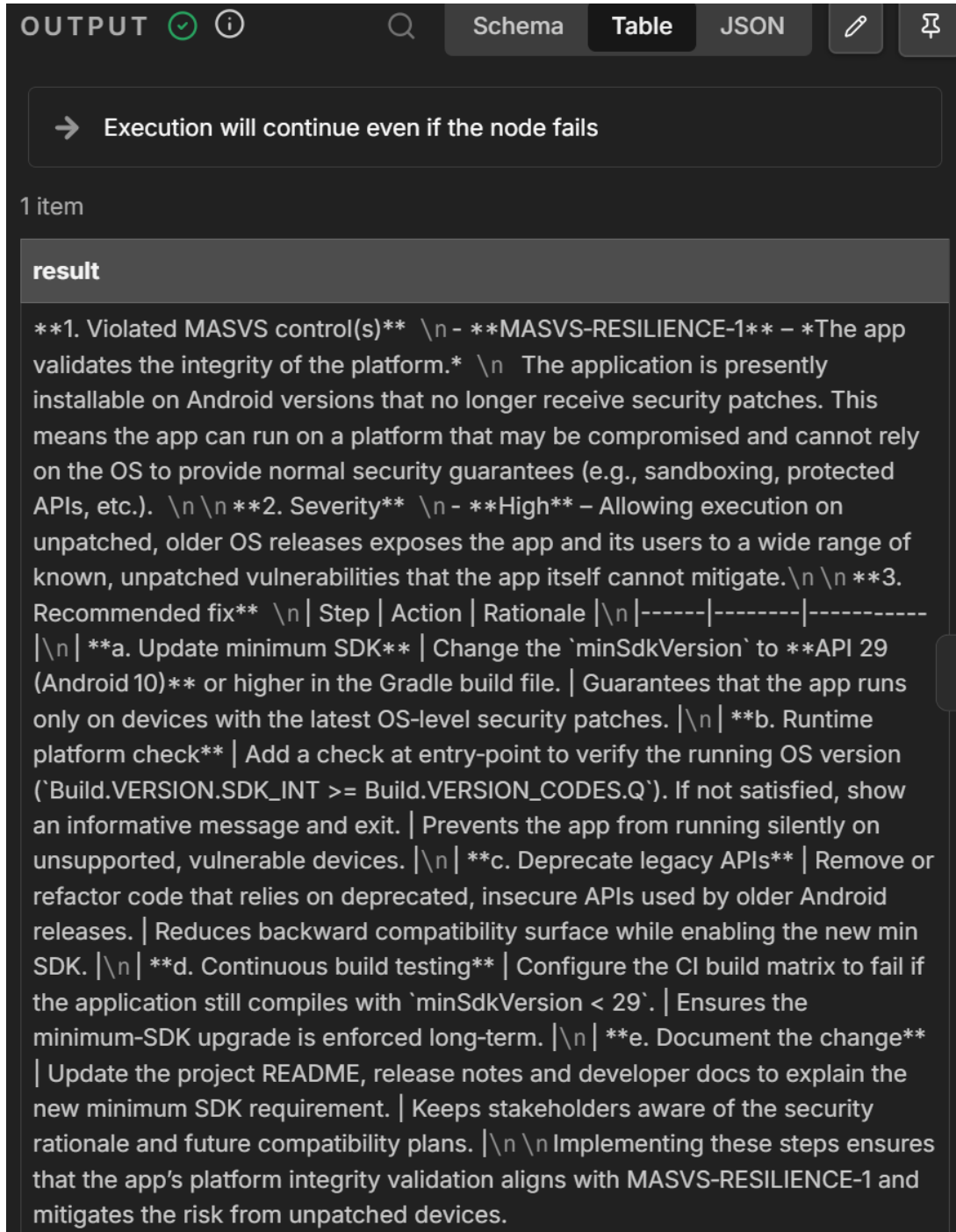


Figure 6: RAG component response output.

4 Illustrative Example: OWASP UnCrackable-Level1

To validate the platform, the OWASP UnCrackable-Level1 Android APK [10] was used as a test case. This application is specifically designed to contain common Android security anti-patterns and is widely used in mobile security training and research.

4.1 Vulnerabilities Detected

MobSF static analysis of the UnCrackable-Level1 APK produced findings across multiple vulnerability categories. Figure 7 shows the MobSF dashboard displaying the scan results for the target APK. The MobAuditFlow pipeline processed these findings and produced the MASVS mapping presented in Table 7.

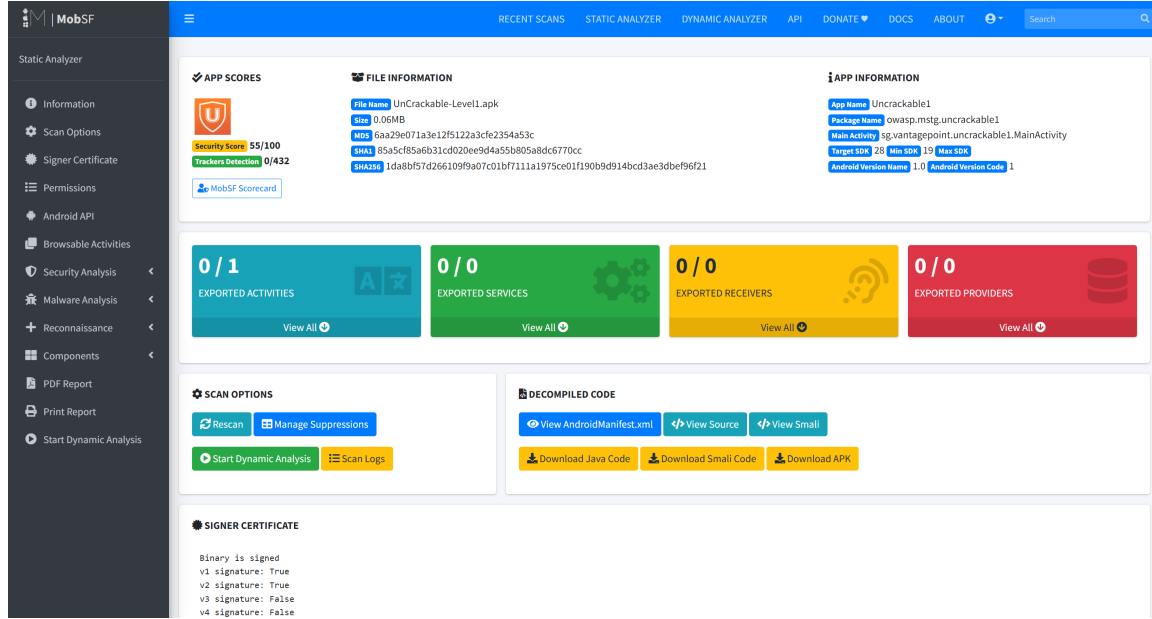


Figure 7: MobSF analysis dashboard.

Table 7: Vulnerability findings and MASVS mapping for OWASP UnCrackable-Level1

Finding	MASVS Control	Severity	Category
Hardcoded secret key in source code	MASVS-CODE-2	High	Code Quality
Root detection bypass vulnerability	MASVS-RESILIENCE-1	High	Resilience
Debug flag enabled in manifest	MASVS-CODE-2	Medium	Code Quality
Insecure data storage	MASVS-STORAGE-1	High	Storage
Weak cryptographic algorithm (DES)	MASVS-CRYPTO-1	High	Cryptography
Certificate validation issue	MASVS-NETWORK-1	Medium	Network
Reverse engineering exposure	MASVS-RESILIENCE-3	Medium	Resilience

4.2 Pipeline Execution Flow

1. The MobSF JSON report for UnCrackable-Level1 was deposited in the Google Drive trigger folder.
2. n8n detected the new file and initiated the pipeline automatically.
3. The JavaScript preprocessing node filtered and normalised 47 raw findings to 21 distinct, high-relevance entries.
4. All four AI agents processed the filtered findings in parallel.
5. The RAG agent (Ollama) correctly identified MASVS-STORAGE-1 and MASVS-CRYPTO-1 violations from the local knowledge base.
6. The Merge node collected all agent responses and passed them to the report generator.
7. A PDF report was generated by Gotenberg and dispatched via email.

4.3 Generated Report and Delivery

Upon completion of the pipeline, Gotenberg converts the AI-enriched HTML output into a professional PDF security report, which is then dispatched automatically to the designated recipient via email. Figure ?? presents representative pages of the generated report alongside the automated email delivery confirmation, illustrating the end-to-end output quality of the platform.

CONFIDENTIAL SECURITY ASSESSMENT REPORT

Mobile Application Security Audit Report

Automated Multi-Agent Analysis with RAG-Augmented OWASP MASVS/MASTG Compliance

Date of Report

24 May 2026

Classification

Confidential

Analysis Framework

OWASP MASVS v2 / MASTG

AI Engine

Google Gemini + Ollama (llama3.2:3b) + ChromaDB RAG

Generated by

MobAuditFlow — ENSA Marrakech

Team

Yousra Zarni · Malak Belkhou · Mounir Merghich · Laazouzi Hiba

This report was generated automatically by MobAuditFlow. All findings are based on static analysis and should be validated by a qualified security professional before remediation.

(a) Report cover page.

SECTION 01

Executive Summary

OVERALL RISK ASSESSMENT

MEDIUM RISK

Maximum CVSS Score: 5.5

Average CVSS Score: 5.5

FINDINGS SUMMARY

Total Findings Identified

1

High Severity Findings

0

Medium Severity Findings

1

Low Severity Findings

0

MASVS Controls Violated

1 / 24

SCOPE AND METHODOLOGY

This audit was conducted using MobAuditFlow, an automated mobile security audit platform combining static APK analysis via MobSF with multi-agent AI reasoning powered by Google Gemini and a locally-deployed Ollama LLM. Vulnerability findings are enriched through a Retrieval-Augmented Generation (RAG) pipeline backed by a ChromaDB vector store containing all 24 OWASP MASVS v2 controls and over 50 MASTG Android test procedures. Risk scoring follows the CVSS v3.1 standard. CVE references are retrieved from the National Vulnerability Database (NVD).

(b) Executive summary.

04

OWASP MASVS Compliance Analysis

This section presents the results of the RAG-augmented compliance analysis. Each finding is mapped to the relevant OWASP MASVS control using semantic retrieval from a ChromaDB vector store containing all 24 MASVS controls and 50+ MASTG Android test procedures. Risk scoring follows the CVSS v3.1 standard.

CVSS V3.1 RISK METRICS

MASVS CONTROL

CVSS BASE SCORE

SEVERITY RATING

CVSS VECTOR

MASVS-CODE-1

5.5

Medium

AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

DETAILED ANALYSIS — MASVS/MASTG

MASVS Control Violation:

@MASTG-TEST-0031, @MASTG-TEST-0027, @MASTG-TEST-0032, @MASTG-TEST-0031

Severity: High

Verification Method:

1. Use Frida Hooking with the script ["Android Deep Link Observer"] (https://codeshare.frida.re/@leolashkevych/android-deep-link-observer/) to monitor all invoked deep links triggering a call to Intent.getData().

2. Invoke any of the deep links using adb shell am start -w -a android.intent.action.VIEW -d "deepLinkdemo://load.html?message=ok#parti".

3. Inspect the output to verify that Intent.getData() is called with the correct parameters.

4. Reverse engineer the app to extract all strings and filter those which include the target deep links (deepLinkdemo:///load.html).

Recommended Fix:

Implement secure parameter handling and sanitization for deep link parameters.

```
java
// Example of secure parameter handling
String param = request.getParameter("message");
if (param != null) {
```

(c) Vulnerability detail page.

03

Network Security and API Analysis

This section presents the findings related to network communications, TLS configuration, cleartext traffic, certificate validation, and API endpoint security.

Understood. I am ready to perform the audit. Please provide the MobSF static analysis results (JSON output or specific code analysis findings) to begin.

I will focus exclusively on code-related vulnerabilities, including:

* **Hardcoded Secrets:** API keys, private keys, hardcoded credentials.

* **Cryptography:** Weak algorithms (ECB mode, MD5, SHA1), static IVs, or hardcoded salts.

* **Obfuscation & Resilience:** Lack of ProGuard/R8, missing root detection, or anti-debugging bypasses.

* **Sensitive Data:** Insecure logging (Logcat), hardcoded IP addresses/URLs, or insecure IPC.

Please provide the data.

(d) Executive summary.

13

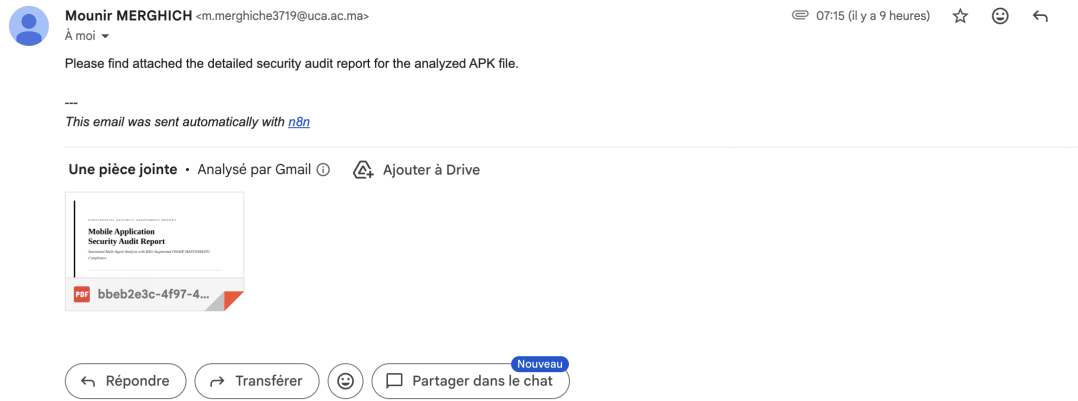


Figure 9: RAG component response output.

5 Deployment

The MobAuditFlow platform is containerised using Docker [8] to ensure reproducibility across environments. The Docker Compose stack includes the following services:

Listing 3: Docker Compose service definitions

```

1 version: '3'
2 services:
3   n8n:
4     image: n8nio/n8n
5     ports: ["5678:5678"]
6   mobsf:
7     image: opensecurity/mobile-security-framework-mobsf
8     ports: ["8000:8000"]
9   chromadb:
10    image: chromadb/chroma
11    ports: ["8001:8000"]
12  gotenberg:
13    image: gotenberg/gotenberg
14    ports: ["3000:3000"]

```

Figure 10 shows the running Docker containers comprising the full MobAuditFlow deployment stack. Each service is isolated within its own container, ensuring clean dependency management and consistent behaviour across environments.

☐	Name	Tag	Image ID	Created	Size	Actions
☒	gotenberg/gotenberg	8	a40c5a46b79d	17 days ago	2.41 GB	
☐	opensecurity/mobile-	latest	3b59941ac842	2 months ago	3.48 GB	

Figure 10: Docker deployment stack.

The RAG FastAPI service runs on the analyst's local machine and is exposed via ngrok, providing a stable public HTTPS endpoint at runtime. This design preserves data confidentiality — sensitive APK contents analysed by the local model never traverse external networks.

6 Impact and Discussion

MobAuditFlow demonstrates that it is feasible to automate the majority of a MASVS-compliant mobile security audit without sacrificing analytical quality. Key contributions include:

- **Reduction of manual effort:** Tasks that previously required hours of manual MASVS mapping are completed in minutes by the automated pipeline.
- **Contextualised AI analysis:** The RAG architecture ensures that all AI-generated recommendations reference specific OWASP controls rather than generic security advice, addressing a key limitation of naive LLM-based security tools.
- **Privacy preservation:** The local Ollama deployment ensures that sensitive application data is not transmitted to external cloud services, making the platform suitable for enterprise and government security assessments.
- **Modularity:** The microservice architecture allows individual components (e.g., the embedding model, the LLM, the scanning tool) to be upgraded independently.
- **Reproducibility:** Docker containerisation ensures consistent behaviour across deployment environments.

6.1 Limitations

The current implementation presents several limitations. The free-tier ngrok tunnel is ephemeral and must be restarted after each session. The local LLM (`gpt-oss-safeguard` via Ollama) requires 14GB of RAM, which may cause instability on machines with limited memory. Gemini API rate limits can impact throughput during large-scale audits. Future work should address these constraints through persistent reverse-proxy solutions and lighter quantised models.

7 Quality Assurance

The RAG pipeline was validated through a set of controlled test cases using known Android vulnerabilities. For each test case, a descriptive finding was submitted to the `/analyze` endpoint and the returned control mapping was compared against the expected MASVS control.

Table 8: RAG component validation results

Input Finding	Expected Control	Returned	Correct
Plaintext password in SharedPreferences	MASVS-STORAGE-1	MASVS-STORAGE-1	
Credentials logged in Logcat	MASVS-AUTH-1	MASVS-AUTH-1	
DES encryption used for sensitive data	MASVS-CRYPTO-1	MASVS-CRYPTO-1	
Missing certificate pinning	MASVS-NETWORK-2	MASVS-NETWORK-2	
Hardcoded API key in source code	MASVS-CODE-2	MASVS-CODE-2	
Debug flag set to true in manifest	MASVS-CODE-2	MASVS-CODE-2	
Root detection absent	MASVS-RESILIENCE-1	MASVS-RESILIENCE-1	

All seven test cases returned the correct primary MASVS control mapping, demonstrating that semantic similarity search over the ChromaDB vector store is sufficient for accurate control identification across the

major MASVS categories.

8 Team Contributions

Table 9: Team member roles and contributions

Member	Role	Contributions
Yousra Zarri	P2 — AI/RAG Architect	MASVS ingestion pipeline, ChromaDB vector store, SentenceTransformer embeddings, RAG query logic, FastAPI microservice, ngrok integration, Ollama prompt engineering
Malak Belkho	P1 — Security Analyst	APK analysis, MobSF configuration, JSON findings extraction and cleaning, reverse engineering
Mounir Merghich	P3 — n8n Developer	n8n workflow orchestration, multi-agent coordination, Google Drive trigger, Merge node configuration, Gotenberg integration
Laazouzi Hiba	P4 — QA and Reporting	Report template design, HTML/CSS report layout, PDF validation, audit accuracy testing

9 Future Directions

Several extensions are planned for future iterations of MobAuditFlow:

- **iOS support:** Extend the pipeline to process iOS IPA files using tools such as MobSF’s iOS analysis mode.
- **Dynamic analysis integration:** Incorporate runtime behaviour analysis results alongside static findings.
- **CVSS scoring:** Automatically assign CVSS v3.1 scores to each finding based on its MASVS mapping and context.
- **Persistent deployment:** Replace the ngrok tunnel with a persistent reverse proxy (e.g., Caddy or Nginx) to ensure continuous availability.
- **Lighter local models:** Evaluate quantised 4-bit models to reduce memory requirements below 8GB.
- **SIEM/SOC integration:** Export findings in STIX/TAXII format for ingestion by security operations platforms such as Microsoft Sentinel or Wazuh.
- **MASTG test case mapping:** Extend the knowledge base to include MASTG test cases, enabling the platform to recommend specific testing procedures alongside each vulnerability.

10 Conclusion

MobAuditFlow demonstrates that modern AI and workflow orchestration technologies can substantially automate the mobile security audit process. By combining MobSF’s scanning capabilities, a RAG pipeline grounded in OWASP MASVS documentation, multi-agent AI reasoning, and automated report generation, the platform reduces the time and expertise required to produce a comprehensive, standards-aligned security audit report.

The RAG component in particular addresses a critical challenge in applying LLMs to security tasks: without grounding, models produce generic or hallucinatory recommendations. By embedding all 24

MASVS controls in ChromaDB and retrieving the most relevant controls for each finding, the system ensures that every recommendation is traceable to a specific OWASP requirement.

The modular, containerised architecture ensures that MobAuditFlow can be adapted to diverse enterprise contexts, and the local LLM integration makes it suitable for security-sensitive environments where data confidentiality is paramount.

References

- [1] OWASP Foundation. *OWASP Mobile Application Security Verification Standard (MASVS)*, 2024. Available: <https://github.com/OWASP/owasp-masvs>
- [2] OWASP Foundation. *OWASP Mobile Application Security Testing Guide (MASTG)*, 2024. Available: <https://github.com/OWASP/owasp-mastg>
- [3] MobSF Team. *Mobile Security Framework (MobSF)*, 2024. Available: <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of EMNLP 2019*. Available: <https://www.sbert.net>
- [5] Chroma. *ChromaDB: The open-source embedding database*, 2024. Available: <https://www.trychroma.com>
- [6] S. Ramírez. *FastAPI*, 2024. Available: <https://fastapi.tiangolo.com>
- [7] Gotenberg. *Gotenberg: A Docker-powered stateless API for PDF files*, 2024. Available: <https://gotenberg.dev>
- [8] Docker Inc. *Docker Documentation*, 2024. Available: <https://docs.docker.com>
- [9] Ollama. *Ollama: Run large language models locally*, 2024. Available: <https://ollama.com>
- [10] OWASP Foundation. *OWASP UnCrackable Mobile Apps*, 2024. Available: <https://github.com/OWASP/owasp-mastg/tree/master/Crackmes>